
Programación en R

de cero a análisis de datos

Curso modular · 8 módulos · ~34 horas

Nivel: principiante → intermedio

Material didáctico con ejercicios, soluciones y rúbricas

Código verificado con R 4.3.3 y tidyverse 2.0.0

25 de septiembre de 2026

Índice general

Presentación del curso	3
¿Para quién es?	3
Objetivos del curso (diseño inverso)	3
Mapa del curso	3
Ponderación sugerida	4
Preparación	4
Estructura de archivos	4
Bibliografía general	5
 1. Módulo 1 — Primeros pasos con R y RStudio	 7
1.1. Contenidos	7
1.2. Desarrollo	7
1.3. Actividades y ejercicios	8
1.4. Evaluación (formativa)	9
1.5. Referencias verificadas	9
 2. Módulo 2 — Estructuras de datos	 10
2.1. Contenidos	10
2.2. Desarrollo	10
2.3. Actividades y ejercicios	11
2.4. Evaluación (formativa)	12
2.5. Referencias verificadas	12
 3. Módulo 3 — Programación: control de flujo y funciones	 13
3.1. Contenidos	13
3.2. Desarrollo	13
3.3. Actividades y ejercicios	14
3.4. Evaluación (sumativa parcial)	15
3.5. Referencias verificadas	15
 4. Módulo 4 — Importar y ordenar datos (tidy data)	 17
4.1. Contenidos	17
4.2. Desarrollo	17
4.3. Actividades y ejercicios	18

4.4. Evaluación (formativa)	19
4.5. Referencias verificadas	19
5. Módulo 5 — Transformar datos con dplyr	20
5.1. Contenidos	20
5.2. Desarrollo	20
5.3. Actividades y ejercicios	21
5.4. Evaluación (sumativa parcial)	22
5.5. Referencias verificadas	22
6. Módulo 6 — Visualización con ggplot2	24
6.1. Contenidos	24
6.2. Desarrollo	24
6.3. Actividades y ejercicios	25
6.4. Evaluación (formativa)	26
6.5. Referencias verificadas	26
7. Módulo 7 — Estadística y modelos en R	27
7.1. Contenidos	27
7.2. Desarrollo	27
7.3. Actividades y ejercicios	28
7.4. Evaluación (sumativa parcial)	29
7.5. Referencias verificadas	29
8. Módulo 8 — Reportes reproducibles con Quarto y proyecto integrador	30
8.1. Contenidos	30
8.2. Desarrollo	30
8.3. Proyecto integrador (capstone)	31
8.4. Referencias verificadas	32

Presentación del curso

Perfil: capacitación en datos/analítica · **Nivel:** principiante → intermedio · **Duración:** 8 módulos, ~34 h · **Modalidad:** autoformación o híbrida (sesiones de 2 h + práctica) · **Idioma:** español

¿Para quién es?

Personas sin experiencia previa en programación (o que vienen de Excel) que quieren usar R para limpiar, analizar, visualizar y reportar datos. Los ejemplos usan un conjunto de **ventas simuladas de retail** (12 tiendas, 3 canales, 4 regiones, 5 productos, 52 semanas de 2025).

Objetivos del curso (diseño inverso)

Al terminar, la persona podrá:

1. **Escribir** scripts de R legibles usando objetos, estructuras de datos, control de flujo y funciones propias.
2. **Importar y ordenar** datos de CSV/Excel siguiendo los principios de *tidy data*.
3. **Transformar** y **resumir** datos con `dplyr` para calcular indicadores por grupo.
4. **Construir** visualizaciones claras con `ggplot2` adecuadas a cada pregunta.
5. **Aplicar e interpretar** estadística descriptiva, pruebas de hipótesis y regresión lineal.
6. **Producir** un reporte reproducible con Quarto que responda una pregunta de negocio.

Evidencia de logro: ejercicios con solución en cada módulo, 3 entregas parciales (módulos 3, 5 y 7) y un **proyecto integrador** evaluado con rúbrica (módulo 8).

Mapa del curso

#	Módulo	Horas	Nivel de Bloom	Evaluación
1	Primeros pasos con R y RStudio	4	Recordar · Aplicar	Quiz
2	Estructuras de datos	4	Comprender · Aplicar	Quiz + práctica
3	Programación: control de flujo y funciones	4	Aplicar · Analizar	Entrega 1 (rúbrica)
4	Importar y ordenar datos	4	Comprender · Aplicar	Quiz + práctica
5	Transformar datos con <code>dplyr</code>	5	Aplicar · Analizar	Entrega 2: mini-reporte de KPIs

#	Módulo	Horas	Nivel de Bloom	Evaluación
6	Visualización con ggplot2	4	Aplicar Evaluar	Revisión entre pares
7	Estadística y modelos	4	Aplicar Evaluar	Entrega 3: informe breve
8	Quarto y proyecto integrador	5	Crear	Proyecto final (rúbrica global)

Ponderación sugerida

Componente	Peso
Quizzes y ejercicios (M1, M2, M4, M6)	15 %
Entrega 1 — funciones (M3)	15 %
Entrega 2 — KPIs con dplyr (M5)	20 %
Entrega 3 — informe estadístico (M7)	15 %
Proyecto integrador (M8)	35 %

Preparación

1. Instala **R** (<https://cran.r-project.org/>) y **RStudio Desktop** (<https://posit.co/download/rstudio-desktop/>).

2. Instala los paquetes del curso:

```
install.packages(c("tidyverse", "readxl", "broom", "scales", "renv"))
```

3. Abre esta carpeta como **Proyecto de RStudio** y genera los datos de práctica (ya incluidos en datos/):

```
source("datos/generar_ventas.R")
```

Todo el código del curso se ejecutó y verificó con R 4.3.3 y tidyverse 2.0.0 usando este directorio como directorio de trabajo.

Estructura de archivos

```
curso-r/
|-- README.md                                # este índice
|-- modulo-01-primeros-pasos.md
|-- modulo-02-estructuras-de-datos.md
|-- modulo-03-programacion.md
|-- modulo-04-importar-y-ordenar.md
```

```

|-- modulo-05-transformar-con-dplyr.md
|-- modulo-06-visualizacion-ggplot2.md
|-- modulo-07-estadistica-y-modelos.md
|-- modulo-08-reportes-quarto-y-proyecto.md
|-- datos/
|   |-- generar_ventas.R           # script reproducible (semilla fija)
|   |-- ventas.csv                 # 3 120 filas: fecha, tienda, producto,
    ↳ precio, unidades
|   |-- tiendas.csv                # 12 tiendas: canal y región
|-- latex/
|   |-- curso-r.tex                # documento principal (portada, índice, preá
    ↳ mbulo)
|   |-- capitulos/                 # un .tex por módulo (generados)
|   |-- convertir.py               # regenera capitulos/ desde los .md (requiere
    ↳ pandoc)
|   |-- curso-r.pdf                # versión compilada

```

Versión LaTeX/PDF: desde `latex/`, ejecuta `python3 convertir.py` (solo si editaste los `.md`) y luego `latexmk -pdf curso-r.tex`. También se puede subir la carpeta `latex/` a Overleaf y compilar con pdfLaTeX.

Bibliografía general

Leyenda: [ES] español · [EN] inglés · [OA] acceso abierto.

Libros

- Grolemund, G. (2014). *Hands-On Programming with R*. O'Reilly Media. ISBN 978-1-4493-5901-0. <https://rstudio-education.github.io/hopr/> [EN] [OA en línea]
- Wickham, H. (2019). *Advanced R* (2.^a ed.). CRC Press. ISBN 978-0-8153-8457-1. <https://adv-r.hadley.nz/> [EN] [OA en línea]
- Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science: Import, tidy, transform, visualize, and model data* (2.^a ed.). O'Reilly Media. ISBN 978-1-4920-9740-2. <https://r4ds.hadley.nz/> [EN] [OA en línea]
 - Traducción al español (2.^a ed.): <https://davidrsch.github.io/r4ds-es/> [ES] [OA]
 - Traducción comunitaria (1.^a ed.): <https://es.r4ds.hadley.nz/> [ES] [OA]

Artículos

- Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10> [EN] [OA]
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686> [EN] [OA]

Documentación

- R Core Team. *An Introduction to R*. <https://cran.r-project.org/doc/manuals/r-release/R-intro.html> [EN] [OA]
- Quarto. *Tutorial: Computations*. <https://quarto.org/docs/get-started/computations/rstudio> [EN]

Videos

- freeCodeCamp.org. *R Programming Tutorial – Learn the Basics of Statistical Computing* [Video] (Barton Poulson, ~2 h). https://youtu.be/_V8eKsto3Ug [EN]
- *Curso completo de R para Data Science con Tidyverse* [Video] (Juan Gabriel Gomila Salas). <https://www.youtube.com/watch?v=9NJyhs5P1GQ> [ES] [*canal y fecha POR VERIFICAR*]
- StatQuest with Josh Starmer. *Linear Regression in R, Step-by-Step* [Video]. https://www.youtube.com/watch?v=u1cc1r_Y7M0 [EN]

Nota de verificación

Las referencias se verificaron mediante búsqueda web (título, autores, editorial, ISBN, DOI y existencia de la URL en índices de búsqueda) el 2026-09-25. No fue posible abrir directamente las páginas desde el entorno de trabajo, por lo que las **fechas de publicación de los videos** y el **nombre exacto del canal** del video en español quedan marcados como *POR VERIFICAR*. Revísalos antes de publicar el curso.

Módulo 1 — Primeros pasos con R y RStudio

- **Duración estimada:** 1.5 h teoría + 2.5 h práctica
- **Objetivos de aprendizaje.** Al finalizar, la persona podrá:
 1. **Identificar** los paneles de RStudio (consola, editor, entorno, archivos/gráficos/ayuda).
 2. **Ejecutar** instrucciones en la consola y desde un script `.R`.
 3. **Crear** objetos con `<-` y **reconocer** sus tipos básicos (`numeric`, `integer`, `character`, `logical`).
 4. **Usar** la ayuda (`?`, `help()`, `example()`) e **instalar** / **cargar** paquetes.

1.1. Contenidos

1. Qué es R y qué es RStudio (el lenguaje vs. el entorno de desarrollo).
2. Instalación: R desde CRAN y RStudio Desktop (Posit).
3. Proyectos de RStudio (`.Rproj`) y el directorio de trabajo.
4. La consola como calculadora; operadores aritméticos y de comparación.
5. Objetos y asignación; reglas de nombres (`snake_case`).
6. Tipos de datos atómicos y `class()`, `typeof()`.
7. Funciones: argumentos por posición y por nombre.
8. Paquetes: `install.packages()` vs. `library()`.
9. Comentarios con `#` y buenas prácticas de scripts.

1.2. Desarrollo

1.2.1. R como calculadora

```
2 + 3 * 4      # precedencia: 14
(2 + 3) * 4    # 20
10 / 3         # 3.333333
10 %/% 3       # división entera: 3
10 %% 3        # residuo (módulo): 1
2^10           # potencia: 1024
sqrt(144)      # 12
```

1.2.2. Objetos y asignación

En R se asigna con `<-` (atajo en RStudio: `Alt + -`). El nombre va a la izquierda.

```
precio <- 4999
unidades <- 12
ingreso <- precio * unidades
ingreso      # 59988
```

1.2.3. Tipos básicos

```
class(3.14)      # "numeric"
class(5L)        # "integer" (la L fuerza entero)
class("Norte")   # "character"
class(TRUE)      # "logical"
is.numeric(precio) # TRUE
as.numeric("42")  # convierte texto a número: 42
```

1.2.4. Funciones y argumentos

```
round(3.14159, digits = 2) # por nombre: 3.14
round(3.14159, 2)         # por posición: 3.14
seq(from = 1, to = 10, by = 3)
```

1.2.5. Ayuda

```
?mean           # abre la ayuda de mean()
help("seq")
example(mean)    # ejecuta los ejemplos de la ayuda
```

1.2.6. Paquetes

Un paquete se **instala una sola vez** por computadora y se **carga en cada sesión**:

```
# install.packages("tidyverse") # una vez (descomenta para instalar)
library(tidyverse)              # en cada sesión
```

Buena práctica: trabaja siempre dentro de un **Proyecto de RStudio** (File → New ↪ Project). Así las rutas son relativas ("datos/ventas.csv") y evitas `setwd()` con rutas absolutas que solo funcionan en tu computadora.

1.3. Actividades y ejercicios

1. Crea un proyecto de RStudio llamado `curso-r` y un script `modulo-01.R`.
2. Calcula el ingreso de vender 35 audífonos de \$899 con 10% de descuento.
3. Guarda tu nombre en un objeto `nombre` y verifica su clase.
4. ¿Qué devuelve `class(5 > 3)`? ¿Y `as.numeric(TRUE)`?
5. Usa la ayuda para averiguar qué hace el argumento `na.rm` de `mean()`.

1.3.1. Soluciones

```
# 2
ingreso_desc <- 35 * 899 * (1 - 0.10)
ingreso_desc           # 28318.5

# 3
```

```

nombre <- "Ana"
class(nombre)           # "character"

# 4
class(5 > 3)            # "logical"
as.numeric(TRUE)        # 1

# 5: na.rm = TRUE elimina los NA antes de calcular
mean(c(10, NA, 20), na.rm = TRUE) # 15

```

1.4. Evaluación (formativa)

Quiz de 5 preguntas:

1. ¿Cuál es la diferencia entre R y RStudio?
2. ¿Qué operador asigna valores a un objeto en el estilo recomendado?
3. ¿Qué devuelve `10 %% 4`?
4. ¿Por qué `install.packages()` no se escribe en cada script?
5. Escribe una línea que calcule la raíz cuadrada de 2 redondeada a 3 decimales.

Respuestas: 1) R es el lenguaje/intérprete; RStudio es el IDE. 2) `<-`. 3) 2. 4) Porque solo se instala una vez; en el script basta `library()`. 5) `round(sqrt(2), 3)`.

1.5. Referencias verificadas

- **Web:** R Core Team. *An Introduction to R*. CRAN. <https://cran.r-project.org/doc/manuals/r-release/R-intro.html> [EN] [OA]
- **Libro:** Grolemund, G. (2014). *Hands-On Programming with R*. O'Reilly Media. ISBN 978-1-4493-5901-0. Versión en línea: <https://rstudio-education.github.io/hopr/> [EN] [OA en línea] — caps. 1–2.
- **Libro:** Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2.^a ed.). O'Reilly Media. ISBN 978-1-4920-9740-2. Traducción al español: <https://davidrsch.github.io/r4ds-es/> [ES] [OA] — cap. “Flujo de trabajo: conceptos básicos”.
- **Video:** freeCodeCamp.org. *R Programming Tutorial – Learn the Basics of Statistical Computing* [Video] (instructor: Barton Poulson). YouTube. https://youtu.be/_V8eKst03Ug [EN] — secciones “Installing R”, “RStudio”, “Packages”. [fecha de publicación POR VERIFICAR]

Módulo 2 — Estructuras de datos

- **Duración estimada:** 1.5 h teoría + 2.5 h práctica
- **Objetivos de aprendizaje.** Al finalizar, la persona podrá:
 1. **Crear** vectores, factores, listas, matrices y data frames.
 2. **Explicar** la coerción de tipos y el reciclaje de vectores.
 3. **Aplicar** indexación por posición, nombre y condición lógica (`[ ]`, `[[ ]]`, `$`).
 4. **Clasificar** qué estructura conviene para un problema dado.

2.1. Contenidos

1. Vectores atómicos: `c()`, `seq()`, `rep()`, `length()`.
2. Operaciones vectorizadas y reciclaje.
3. Coerción: jerarquía `logical < integer < numeric < character`.
4. Valores especiales: NA, NULL, NaN, Inf.
5. Factores: niveles y orden.
6. Matrices: `matrix()`, `dim()`.
7. Listas: estructuras heterogéneas.
8. Data frames: la tabla de datos; `str()`, `head()`, `summary()`, `nrow()`.
9. Indexación: `[ ]`, `[[ ]]`, `$` y filtros lógicos.

2.2. Desarrollo

2.2.1. Vectores y vectorización

```
unidades <- c(12, 7, 20, 3, 15)
precios  <- c(4999, 7999, 899, 6499, 2499)
ingresos <- unidades * precios      # operación elemento a elemento
sum(ingresos)
mean(unidades)
length(unidades)
1:10 * 2                          # reciclaje de un escalar
```

2.2.2. Coerción y NA

```
c(1, "dos", TRUE)      # todo se vuelve character
c(1, TRUE, FALSE)      # 1 1 0 (logical -> numeric)
x <- c(4, NA, 10)
sum(x)                 # NA
sum(x, na.rm = TRUE)   # 14
is.na(x)               # FALSE TRUE FALSE
```

2.2.3. Factores

```
canal <- factor(c("En línea", "Departamental", "En línea", "Autoservicio"))
levels(canal)                                # orden alfabético por defecto
table(canal)
talla <- factor(c("M", "S", "L", "M"), levels = c("S", "M", "L"), ordered = TRUE)
talla < "L"
```

2.2.4. Matrices y listas

```
m <- matrix(1:6, nrow = 2)
dim(m)
m[2, 3]                                     # fila 2, columna 3

tienda <- list(id = "T01", region = "Norte", ventas = c(120, 98, 143))
tienda$region
tienda[["ventas"]][2]                      # 98
str(tienda)
```

2.2.5. Data frames

```
df <- data.frame(
  producto = c("Smartphone A", "Tablet", "Audífonos"),
  precio   = c(4999, 6499, 899),
  unidades = c(12, 4, 20)
)
str(df)
df$ingreso <- df$precio * df$unidades      # nueva columna
df[df$unidades > 10, ]                    # filas que cumplen condición
df[, c("producto", "ingreso")]            # columnas por nombre
nrow(df); ncol(df)
```

2.2.6. Leer el conjunto de datos del curso (R base)

```
ventas <- read.csv("datos/ventas.csv")
head(ventas)
summary(ventas$unidades)
```

2.3. Actividades y ejercicios

1. Crea un vector con las temperaturas `c(22, 25, NA, 30, 28)` y calcula su promedio ignorando el NA.
2. Con `ventas`, ¿cuántas filas y columnas tiene? ¿Cuántos NA hay en `unidades`?
3. Convierte `ventas$producto` en factor y muestra cuántos registros hay por producto.
4. Extrae las filas de `ventas` del producto "Tablet" con más de 8 unidades.
5. Crea una lista `resumen` con el total de unidades y el producto más frecuente.

2.3.1. Soluciones

```
# 1
temp <- c(22, 25, NA, 30, 28)
mean(temp, na.rm = TRUE)           # 26.25

# 2
dim(ventas)
sum(is.na(ventas$unidades))        # 40

# 3
ventas$producto <- factor(ventas$producto)
table(ventas$producto)

# 4
tablets <- ventas[ventas$producto == "Tablet" &
                  !is.na(ventas$unidades) &
                  ventas$unidades > 8, ]
head(tablets)

# 5
resumen <- list(
  total_unidades = sum(ventas$unidades, na.rm = TRUE),
  mas_frecuente  = names(which.max(table(ventas$producto)))
)
str(resumen)
```

2.4. Evaluación (formativa)

- **Quiz:** ¿Qué devuelve `c(1, "a")`? ¿Cuál es la diferencia entre `lista["x"]` y `lista[["x"]]`? ¿Para qué sirve `levels=` en `factor()`?
- **Práctica calificada:** script que lea `ventas.csv`, cree la columna ingreso y muestre las 5 filas con mayor ingreso (pista: `order()`, con `decreasing = TRUE`).

2.5. Referencias verificadas

- **Libro:** Grolemund, G. (2014). *Hands-On Programming with R*. O'Reilly Media. ISBN 978-1-4493-5901-0. <https://rstudio-education.github.io/hopr/> [EN] [OA en línea] — cap. “R Objects” y “R Notation”.
- **Libro:** Wickham, H. (2019). *Advanced R* (2.^a ed.). CRC Press. ISBN 978-0-8153-8457-1. <https://adv-r.hadley.nz/> [EN] [OA en línea, CC BY-NC-SA 4.0] — cap. 3 “Vectors” y cap. 4 “Subsetting” (lectura de profundización).
- **Web:** R Core Team. *An Introduction to R* — secciones “Simple manipulations; numbers and vectors” y “Lists and data frames”. <https://cran.r-project.org/doc/manuals/r-release/R-intro.html> [EN] [OA]
- **Video:** freeCodeCamp.org. *R Programming Tutorial – Learn the Basics of Statistical Computing* [Video]. YouTube. https://youtu.be/_V8eKsto3Ug [EN] — secciones “Data Formats”, “Factors”, “Entering Data”, “Importing Data”.

Módulo 3 — Programación: control de flujo y funciones

- **Duración estimada:** 1.5 h teoría + 2.5 h práctica
- **Objetivos de aprendizaje.** Al finalizar, la persona podrá:
 1. Implementar decisiones con `if/else`, `ifelse()` y `dplyr::case_when()`.
 2. Ejecutar iteraciones con `for` y `while`.
 3. Escribir funciones propias con argumentos, valores por defecto y validaciones.
 4. Comparar un bucle con la alternativa vectorizada o funcional (`sapply()`, `purrr::map` $\hookrightarrow$ `()`).

3.1. Contenidos

1. Operadores lógicos: `&`, `|`, `!`, `&&`, `||`, `%in%`.
2. `if`, `else if`, `else`; `ifelse()` vectorizado.
3. Bucles `for` y `while`; `break` y `next`.
4. Anatomía de una función: nombre, argumentos, cuerpo, valor de retorno.
5. Valores por defecto y validación con `stop()`.
6. Alcance (scope) de variables.
7. Vectorización y familia `apply` (`sapply`, `lapply`) vs. `purrr::map()`.

3.2. Desarrollo

3.2.1. Condicionales

```
unidades <- 25
if (unidades >= 20) {
  "Alta rotación"
} else if (unidades >= 10) {
  "Rotación media"
} else {
  "Baja rotación"
}

u <- c(3, 12, 25)
ifelse(u >= 10, "Suficiente", "Reabastecer") # vectorizado
"Tablet" %in% c("Tablet", "Reloj")          # TRUE
```

3.2.2. Bucles

```
productos <- c("Smartphone A", "Tablet", "Reloj")
for (p in productos) {
  print(paste("Revisando inventario de", p))
}
```

```

}

stock <- 50
semanas <- 0
while (stock > 0) {
  stock <- stock - 12
  semanas <- semanas + 1
}
semanas          # semanas hasta agotar: 5

```

3.2.3. Funciones

```

ingreso <- function(precio, unidades, descuento = 0) {
  if (descuento < 0 || descuento >= 1) {
    stop("`descuento` debe estar en [0, 1).")
  }
  precio * unidades * (1 - descuento)
}

ingreso(4999, 10)
ingreso(4999, 10, descuento = 0.15)
ingreso(c(899, 2499), c(20, 8))      # funciona con vectores

```

3.2.4. Vectorizar en lugar de iterar

```

x <- 1:5
# Con bucle
cuadrados <- numeric(length(x))
for (i in seq_along(x)) cuadrados[i] <- x[i]^2
cuadrados
# Vectorizado (preferido)
x^2
# Funcional
sapply(x, function(v) v^2)
library(purrr)
map_dbl(x, \(v) v^2)          # \(v) es la sintaxis corta de function(v), R >=
                               ↪ 4.1

```

3.3. Actividades y ejercicios

1. Escribe `clasificar_rotacion(u)` que devuelva "Alta", "Media" o "Baja" para un **vector** de unidades (umbrales 20 y 10).
2. Escribe `margen(precio, costo)` que devuelva el margen porcentual y lance error si `costo > precio`.
↪ `precio`.
3. Con un bucle `for`, imprime el total de unidades por producto en `ventas`. Luego obtén lo mismo con `tapply()`.
4. Simula lanzar un dado 1 000 veces con `sample()` y calcula la proporción de seises.

3.3.1. Soluciones

```

# 1 (dplyr::case_when es vectorizado y legible)
clasificar_rotacion <- function(u) {
  dplyr::case_when(
    u >= 20 ~ "Alta",
    u >= 10 ~ "Media",
    TRUE   ~ "Baja"
  )
}
clasificar_rotacion(c(3, 12, 25))

# 2
margen <- function(precio, costo) {
  if (any(costo > precio)) stop("El costo no puede ser mayor que el precio.")
  (precio - costo) / precio * 100
}
margen(100, 60) # 40

# 3
ventas <- read.csv("datos/ventas.csv")
for (p in unique(ventas$producto)) {
  total <- sum(ventas$unidades[ventas$producto == p], na.rm = TRUE)
  cat(p, ":", total, "\n")
}
tapply(ventas$unidades, ventas$producto, sum, na.rm = TRUE)

# 4
set.seed(1)
dado <- sample(1:6, 1000, replace = TRUE)
mean(dado == 6)

```

3.4. Evaluación (sumativa parcial)

Entrega un script `funciones_ventas.R` con 3 funciones documentadas (comentario de propósito, argumentos y retorno) y al menos una validación con `stop()`. Rúbrica:

Criterio	3 — Logrado	2 — En proceso	1 — Inicial
Correctitud	Las 3 funciones dan el resultado esperado	2 funciones correctas	≤ 1 correcta
Vectorización	Aceptan vectores sin bucles innecesarios	Parcial	Solo escalares
Validación	Mensajes de error claros	Validación incompleta	Sin validación
Estilo	Nombres claros, comentarios útiles	Aceptable	Difícil de leer

3.5. Referencias verificadas

- **Libro:** Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2.^a ed.). O'Reilly Media. ISBN 978-1-4920-9740-2. Español: <https://davidrsch.github.io>

- [.io/r4ds-es/](https://rstudio-education.github.io/r4ds-es/) [ES] [OA] — caps. “Funciones” e “Iteración”.
- **Libro:** Golemund, G. (2014). *Hands-On Programming with R*. O’Reilly Media. ISBN 978-1-4493-5901-0. <https://rstudio-education.github.io/hopr/> [EN] [OA en línea] — proyecto “Slot machine” (if/else, bucles, funciones).
 - **Libro:** Wickham, H. (2019). *Advanced R* (2.^a ed.). CRC Press. ISBN 978-0-8153-8457-1. <https://adv-r.hadley.nz/> [EN] [OA en línea] — cap. 5 “Control flow” y cap. 6 “Functions”.

Módulo 4 — Importar y ordenar datos (tidy data)

- **Duración estimada:** 1.5 h teoría + 2.5 h práctica
- **Objetivos de aprendizaje.** Al finalizar, la persona podrá:
 1. **Explicar** los tres principios de los datos ordenados (*tidy data*).
 2. **Importar** archivos CSV y Excel con `readr` y `readxl`, controlando tipos de columna.
 3. **Transformar** tablas entre formato ancho y largo con `pivot_longer()` / `pivot_wider()`.
 4. **Diferenciar** los problemas de calidad más comunes (tipos erróneos, NA, duplicados).

4.1. Contenidos

1. El *tidyverse* y los *tibbles*.
2. Principios de datos ordenados: cada variable una columna, cada observación una fila, cada valor una celda.
3. `readr::read_csv()`, especificación de tipos con `col_types`, `problems()`.
4. `readxl::read_excel()`: hojas y rangos.
5. `tidyr::pivot_longer()` y `pivot_wider()`.
6. `separate_wider_delim()` y `unite()`.
7. Manejo de NA: `drop_na()`, `replace_na()`; duplicados con `distinct()`.
8. Exportar: `write_csv()`.

4.2. Desarrollo

4.2.1. Importar con readr

```
library(tidyverse)

ventas <- read_csv("datos/ventas.csv", col_types = cols(
  fecha      = col_date(),
  tienda_id  = col_character(),
  producto   = col_character(),
  precio     = col_double(),
  unidades   = col_integer()
))
ventas
glimpse(ventas)
tiendas <- read_csv("datos/tiendas.csv", show_col_types = FALSE)
```

4.2.2. Excel

```
library(readxl)
# excel_sheets("datos/archivo.xlsx")           # lista las hojas
# read_excel("datos/archivo.xlsx", sheet = "Ventas", range = "A1:E500")
```

4.2.3. De ancho a largo y de regreso

Muchas hojas de cálculo guardan un mes por columna: eso **no** es *tidy*, porque “mes” es una variable.

```
reporte_ancho <- tribble(
  ~tienda, ~ene, ~feb, ~mar,
  "T01",    120,   98,  143,
  "T02",    87,  110,   95
)

reporte_largo <- reporte_ancho |>
  pivot_longer(cols = ene:mar, names_to = "mes", values_to = "unidades")
reporte_largo

reporte_largo |>
  pivot_wider(names_from = mes, values_from = unidades)
```

|> es el *pipe* nativo de R (≥ 4.1): $x \mid> f(y)$ equivale a $f(x, y)$. Léelo como “y luego”.

4.2.4. Separar y unir columnas

```
codigos <- tibble(sku = c("CEL-001-NEGRO", "TAB-014-GRIS"))
codigos |>
  separate_wider_delim(sku, delim = "-", names = c("categoria", "numero", "color"))
```

4.2.5. Valores faltantes y duplicados

```
ventas |> summarise(faltantes = sum(is.na(unidades)))
ventas_sin_na <- ventas |> drop_na(unidades)
ventas |> mutate(unidades = replace_na(unidades, 0L)) |> head()
ventas |> distinct(tienda_id, fecha, producto) |> nrow() # verifica llave única
```

4.3. Actividades y ejercicios

1. Importa `ventas.csv` con `read_csv()` sin `col_types` y compara el tipo que asigna a `unidades` y `fecha` con la versión especificada.
2. Convierte `ventas` a formato ancho: una fila por `fecha` y `tienda_id`, una columna por producto con las unidades.
3. Regresa el resultado anterior a formato largo y comprueba que recuperas el mismo número de filas.
4. Cuenta los NA de unidades por producto.

4.3.1. Soluciones

```
# 1
v2 <- read_csv("datos/ventas.csv", show_col_types = FALSE)
spec(v2)           # readr infiere fecha como date y unidades como double

# 2
ancho <- ventas |>
  select(fecha, tienda_id, producto, unidades) |>
  pivot_wider(names_from = producto, values_from = unidades)
ancho

# 3
largo <- ancho |>
  pivot_longer(-c(fecha, tienda_id), names_to = "producto", values_to = "unidades")
nrow(largo) == nrow(ventas)    # TRUE

# 4
ventas |>
  group_by(producto) |>
  summarise(n_na = sum(is.na(unidades)))
```

4.4. Evaluación (formativa)

- **Quiz:** Da un ejemplo de tabla que viole cada principio de *tidy data*. ¿Qué función usarías para arreglarla?
- **Práctica:** Recibe un Excel “reporte mensual” con meses en columnas y entrégalo en formato largo como CSV con `write_csv()`.

4.5. Referencias verificadas

- **Artículo:** Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10> [EN] [OA]
- **Artículo:** Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686> [EN] [OA, CC BY 4.0]
- **Libro:** Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2.^a ed.). O'Reilly Media. ISBN 978-1-4920-9740-2. Español: <https://davidrsch.github.io/r4ds-es/> [ES] [OA] — caps. “Ordenar datos”, “Importación de datos”, “Hojas de cálculo”.
- **Video:** Gomila Salas, J. G. (Frogames). *Curso completo de R para Data Science con Tidyverse – R for Data Science en Español* [Video]. YouTube. <https://www.youtube.com/watch?v=9NJyhs5PlGQ> [ES] [nombre exacto del canal y fecha POR VERIFICAR]

Módulo 5 — Transformar datos con dplyr

- **Duración estimada:** 1.5 h teoría + 3.5 h práctica
- **Objetivos de aprendizaje.** Al finalizar, la persona podrá:
 1. **Aplicar** los verbos de dplyr (`filter`, `select`, `mutate`, `arrange`, `summarise`) encadenados con `|>`.
 2. **Calcular** indicadores agregados por grupo con `group_by()` / `.by`.
 3. **Combinar** tablas con uniones (`left_join`, `inner_join`, `anti_join`).
 4. **Usar** `lubridate` y `stringr` para trabajar con fechas y texto.
 5. **Analizar** el desempeño por canal, región y producto a partir de datos crudos.

5.1. Contenidos

1. Los cinco verbos principales y el *pipe*.
2. `mutate()` con `if_else()` y `case_when()`.
3. `group_by()` + `summarise()`; `count()`; agrupación temporal con `.by`.
4. Funciones de ventana: `lag()`, `cumsum()`, `rank()`, `slice_max()`.
5. Uniones de tablas y llaves.
6. Fechas con `lubridate`: `year()`, `month()`, `floor_date()`.
7. Texto con `stringr`: `str_detect()`, `str_to_upper()`, `str_replace()`.

5.2. Desarrollo

```
library(tidyverse)
ventas <- read_csv("datos/ventas.csv", show_col_types = FALSE)
tiendas <- read_csv("datos/tiendas.csv", show_col_types = FALSE)
```

5.2.1. Los verbos básicos

```
ventas |>
  filter(producto == "Tablet", unidades >= 8) |>
  select(fecha, tienda_id, unidades) |>
  arrange(desc(unidades))

ventas <- ventas |>
  mutate(ingreso = precio * unidades)
```

5.2.2. Resúmenes por grupo

```

ventas |>
  group_by(producto) |>
  summarise(
    unidades = sum(unidades, na.rm = TRUE),
    ingreso = sum(ingreso, na.rm = TRUE),
    ticket_promedio = ingreso / unidades
  ) |>
  arrange(desc(ingreso))

ventas |> count(producto)                                # conteo rápido

```

5.2.3. Uniones

```

ventas_det <- ventas |>
  left_join(tiendas, by = "tienda_id")

ventas_det |>
  summarise(ingreso = sum(ingreso, na.rm = TRUE), .by = c(canal, region)) |>
  arrange(canal, desc(ingreso))

# ¿Hay ventas de tiendas que no existen en el catálogo?
ventas |> anti_join(tiendas, by = "tienda_id") |> nrow()    # 0

```

5.2.4. Fechas

```

mensual <- ventas_det |>
  mutate(mes = floor_date(fecha, "month")) |>
  summarise(ingreso = sum(ingreso, na.rm = TRUE), .by = c(mes, canal)) |>
  arrange(canal, mes) |>
  group_by(canal) |>
  mutate(var_pct = (ingreso / lag(ingreso) - 1) * 100) |>
  ungroup()
mensual

```

5.2.5. Texto

```

ventas_det |>
  filter(str_detect(producto, "Smartphone")) |>
  mutate(producto = str_to_upper(producto)) |>
  distinct(producto)

```

5.3. Actividades y ejercicios

1. ¿Cuál fue la **tienda** con mayor ingreso anual? Muestra el top 3 con su canal y región.
2. Calcula la **participación** (%) de cada canal en el ingreso total.
3. Para cada producto, encuentra la **semana** de mayores unidades vendidas (`slice_max()`).
4. Compara el ingreso promedio semanal de **noviembre–diciembre** vs. el resto del año por producto.
5. Reto: con `anti_join()`, verifica que cada combinación tienda–semana tenga los 5 productos.

5.3.1. Soluciones

```
# 1
ventas_det |>
  summarise(ingreso = sum(ingreso, na.rm = TRUE), .by = c(tienda_id, canal, region))
  ↪ |>
  slice_max(ingreso, n = 3)

# 2
ventas_det |>
  summarise(ingreso = sum(ingreso, na.rm = TRUE), .by = canal) |>
  mutate(participacion = ingreso / sum(ingreso) * 100)

# 3
ventas |>
  summarise(unidades = sum(unidades, na.rm = TRUE), .by = c(producto, fecha)) |>
  group_by(producto) |>
  slice_max(unidades, n = 1, with_ties = FALSE)

# 4
ventas |>
  mutate(temporada = if_else(month(fecha) %in% 11:12, "Nov-Dic", "Resto")) |>
  summarise(ingreso_sem = sum(ingreso, na.rm = TRUE), .by = c(producto, temporada,
    ↪ fecha)) |>
  summarise(promedio = mean(ingreso_sem), .by = c(producto, temporada)) |>
  pivot_wider(names_from = temporada, values_from = promedio) |>
  mutate(incremento_pct = (`Nov-Dic` / Resto - 1) * 100)

# 5
esperado <- expand_grid(
  tienda_id = unique(ventas$tienda_id),
  fecha      = unique(ventas$fecha),
  producto   = unique(ventas$producto)
)
esperado |> anti_join(ventas, by = c("tienda_id", "fecha", "producto")) |> nrow() #
  ↪ 0
```

5.4. Evaluación (sumativa parcial)

Mini-reporte de KPIs: script que produzca una tabla con ingreso, unidades y ticket promedio por canal y región, más la variación mensual por canal. Se evalúa: resultados correctos (40 %), uso idiomático de dplyr (30 %), legibilidad (30 %).

5.5. Referencias verificadas

- **Libro:** Wickham, H., Çetinkaya-Rundel, M., & Golemund, G. (2023). *R for Data Science* (2.^a ed.). O'Reilly Media. ISBN 978-1-4920-9740-2. Español: <https://davidrsch.github.io/r4ds-es/> [ES] [OA] — caps. “Transformación de datos”, “Uniones”, “Fechas y horas”, “Cadenas”.
- **Web:** R para Ciencia de Datos (traducción de la 1.^a ed. por la comunidad latinoamericana de R). <https://es.r4ds.hadley.nz/> [ES] [OA] — alternativa en español.
- **Artículo:** Wickham, H., et al. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686> [EN] [OA]

- **Video:** Gomila Salas, J. G. (Frogames). *Curso completo de R para Data Science con Tidyverse* [Video]. YouTube. <https://www.youtube.com/watch?v=9NJyhs5P1GQ> [ES]
[canal y fecha POR VERIFICAR]

Módulo 6 — Visualización con ggplot2

- **Duración estimada:** 1.5 h teoría + 2.5 h práctica
- **Objetivos de aprendizaje.** Al finalizar, la persona podrá:
 1. **Explicar** la gramática de gráficos: datos, estéticas (`aes`), geometrías, escalas, facetas y temas.
 2. **Construir** gráficos de barras, líneas, dispersión, histogramas y cajas.
 3. **Seleccionar** el tipo de gráfico adecuado según la pregunta (comparar, tendencia, distribución, relación).
 4. **Producir** gráficos listos para reporte (títulos, etiquetas, formato de ejes) y exportarlos con `ggsave()`.

6.1. Contenidos

1. La gramática de gráficos por capas.
2. `aes()`: x, y, color, fill, size; estéticas fijas vs. mapeadas.
3. Geometrías: `geom_col`, `geom_line`, `geom_point`, `geom_histogram`, `geom_boxplot`.
4. Facetas: `facet_wrap()`.
5. Escalas y formato: `scales::label_dollar()`, `label_percent()`.
6. Etiquetas y temas: `labs()`, `theme_minimal()`.
7. Exportar: `ggsave()`.

6.2. Desarrollo

```
library(tidyverse)
ventas <- read_csv("datos/ventas.csv", show_col_types = FALSE) |>
  mutate(ingreso = precio * unidades)
tiendas <- read_csv("datos/tiendas.csv", show_col_types = FALSE)
ventas_det <- left_join(ventas, tiendas, by = "tienda_id")
```

6.2.1. Comparar categorías: barras

```
por_producto <- ventas |>
  summarise(ingreso = sum(ingreso, na.rm = TRUE), .by = producto)

ggplot(por_producto, aes(x = ingreso, y = fct_reorder(producto, ingreso))) +
  geom_col(fill = "steelblue") +
  scale_x_continuous(labels = scales::label_dollar(scale = 1e-6, suffix = " M")) +
  labs(title = "Ingreso anual por producto", x = "Ingreso (MXN)", y = NULL) +
  theme_minimal()
```

Barras **horizontales** y **ordenadas** facilitan leer nombres largos y comparar magnitudes.

6.2.2. Tendencia: líneas

```
semanal <- ventas_det |>
  summarise(ingreso = sum(ingreso, na.rm = TRUE), .by = c(fecha, canal))

g_tendencia <- ggplot(semanal, aes(fecha, ingreso, color = canal)) +
  geom_line(linewidth = 0.8) +
  scale_y_continuous(labels = scales::label_dollar()) +
  labs(title = "Ingreso semanal por canal, 2025", x = NULL, y = "Ingreso", color = "
  ↪ Canal") +
  theme_minimal()
g_tendencia
```

6.2.3. Distribución: histograma y cajas

```
ggplot(ventas, aes(unidades)) +
  geom_histogram(binwidth = 2, fill = "grey40", color = "white") +
  facet_wrap(~ producto, scales = "free_y") +
  labs(title = "Distribución de unidades semanales por producto")

ggplot(ventas_det, aes(region, unidades, fill = region)) +
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Unidades por región", x = NULL)
```

Los NA de unidades se descartan con una advertencia (“Removed rows...”). Es informativa, no un error.

6.2.4. Relación: dispersión

```
por_tienda <- ventas_det |>
  summarise(unidades = sum(unidades, na.rm = TRUE),
            ingreso = sum(ingreso, na.rm = TRUE), .by = c(tienda_id, canal))

ggplot(por_tienda, aes(unidades, ingreso, color = canal)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = FALSE, color = "black", linewidth = 0.5) +
  labs(title = "Unidades vs. ingreso por tienda")
```

6.2.5. Exportar

```
ggsave("tendencia_canal.png", g_tendencia, width = 8, height = 4.5, dpi = 300)
```

6.3. Actividades y ejercicios

1. Gráfico de barras apiladas al 100 % de la participación de cada producto dentro de cada canal (`position = "fill"`).
2. Líneas de unidades mensuales por producto con `facet_wrap()`.
3. Mejora un gráfico “por defecto” aplicando: orden, título que afirme la conclusión, ejes formateados y sin leyenda redundante.
4. Exporta tu mejor gráfico a PNG de 300 dpi.

6.3.1. Soluciones

```
# 1
ventas_det |>
  summarise(ingreso = sum(ingreso, na.rm = TRUE), .by = c(canal, producto)) |>
  ggplot(aes(canal, ingreso, fill = producto)) +
  geom_col(position = "fill") +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(title = "Mezcla de productos por canal", x = NULL, y = "Participación", fill
    ↪ = NULL)

# 2
ventas |>
  mutate(mes = floor_date(fecha, "month")) |>
  summarise(unidades = sum(unidades, na.rm = TRUE), .by = c(mes, producto)) |>
  ggplot(aes(mes, unidades)) +
  geom_line() +
  facet_wrap(~ producto, scales = "free_y") +
  labs(title = "Las ventas de todos los productos suben en noviembre y diciembre",
    x = NULL, y = "Unidades")
```

6.4. Evaluación (formativa)

Galería de pares: cada persona presenta un gráfico y recibe retroalimentación con la lista de verificación: ¿responde una pregunta clara?, ¿el tipo de gráfico es adecuado?, ¿ejes y unidades legibles?, ¿el título dice la conclusión?, ¿colores con propósito?

6.5. Referencias verificadas

- **Libro:** Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2.^a ed.). O'Reilly Media. ISBN 978-1-4920-9740-2. Español: <https://davidrsch.github.io/r4ds-es/> [ES] [OA] — caps. “Visualización de datos”, “Capas”, “Análisis exploratorio”, “Comunicación”.
- **Web:** Wickham, H., Navarro, D., & Pedersen, T. L. *ggplot2: Elegant Graphics for Data Analysis* (3.^a ed., versión en línea). <https://ggplot2-book.org/> [EN] [OA en línea] [datos de edición impresa POR VERIFICAR]
- **Video:** freeCodeCamp.org. *R Programming Tutorial – Learn the Basics of Statistical Computing* [Video]. YouTube. https://youtu.be/_V8eKsto3Ug [EN] — secciones de gráficos (`plot()`, barras, histogramas, dispersión) en R base, útil para contrastar con ggplot2.

Módulo 7 — Estadística y modelos en R

- **Duración estimada:** 1.5 h teoría + 2.5 h práctica
- **Objetivos de aprendizaje.** Al finalizar, la persona podrá:
 1. **Calcular** estadísticos descriptivos (media, mediana, desviación estándar, cuantiles, correlación).
 2. **Ejecutar** pruebas de hipótesis básicas (`t.test()`, `wilcox.test()`, `chisq.test()`).
 3. **Ajustar** un modelo de regresión lineal con `lm()` usando la sintaxis de fórmulas.
 4. **Interpretar** coeficientes, valores p, R^2 y residuos, y **justificar** si el modelo es adecuado.

7.1. Contenidos

1. Estadística descriptiva por grupo con `dplyr`.
2. Correlación: `cor()` (Pearson y Spearman).
3. Pruebas de hipótesis: t de Welch, Mann-Whitney/Wilcoxon, chi-cuadrada.
4. Fórmulas en R: $y \sim x_1 + x_2$.
5. Regresión lineal: `lm()`, `summary()`, `coef()`, `confint()`, `predict()`.
6. Diagnóstico con residuos; resultados ordenados con `broom::tidy()`.

7.2. Desarrollo

```
library(tidyverse)
ventas <- read_csv("datos/ventas.csv", show_col_types = FALSE)
tiendas <- read_csv("datos/tiendas.csv", show_col_types = FALSE)
ventas_det <- ventas |>
  left_join(tiendas, by = "tienda_id") |>
  drop_na(unidades) |>
  mutate(temporada = if_else(month(fecha) %in% 11:12, "Nov-Dic", "Resto"))
```

7.2.1. Descriptivos

```
ventas_det |>
  summarise(
    media = mean(unidades),
    mediana = median(unidades),
    de = sd(unidades),
    p90 = quantile(unidades, 0.9),
    .by = producto
  )
```

7.2.2. Prueba t: ¿la temporada cambia las ventas?

```
reloj <- ventas_det |> filter(producto == "Reloj")
t.test(unidades ~ temporada, data = reloj)      # Welch por defecto
wilcox.test(unidades ~ temporada, data = reloj) # alternativa no paramétrica
```

Cuidado: las observaciones semanales de una misma tienda no son del todo independientes. Aquí la prueba es ilustrativa; en un análisis formal hay que considerar la autocorrelación.

7.2.3. Chi-cuadrada: ¿la mezcla de productos depende del canal?

```
tabla <- xtabs(unidades ~ canal + producto, data = ventas_det)
tabla
chisq.test(tabla)
```

7.2.4. Regresión lineal

```
semanal <- ventas_det |>
  summarise(unidades = sum(unidades), .by = c(fecha, canal, temporada))

modelo <- lm(unidades ~ temporada + canal, data = semanal)
summary(modelo)
confint(modelo)

library(broom)
tidy(modelo)      # coeficientes como tibble
glance(modelo)    # R2, AIC, etc.

nuevo <- tibble(temporada = "Nov-Dic", canal = "En línea")
predict(modelo, newdata = nuevo, interval = "prediction")
```

7.2.5. Diagnóstico

```
aug <- augment(modelo)
ggplot(aug, aes(.fitted, .resid)) +
  geom_point(alpha = 0.5) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(title = "Residuos vs. ajustados", x = "Ajustado", y = "Residuo")
```

7.3. Actividades y ejercicios

1. Calcula la matriz de correlación entre precio y unidades a nivel registro. ¿Qué signo esperas y por qué?
2. Prueba si las unidades de **Audífonos** difieren entre las regiones **Norte** y **Sur**.
3. Ajusta `lm(unidades ~ temporada * canal)` y compara su R^2 ajustado con el modelo sin interacción.
4. Redacta en 3 líneas la interpretación del coeficiente `temporadaResto` para un público no técnico.

7.3.1. Soluciones

```
# 1: negativo; los productos caros se venden en menos unidades
cor(ventas_det$precio, ventas_det$unidades)
cor(ventas_det$precio, ventas_det$unidades, method = "spearman")

# 2
audif <- ventas_det |> filter(producto == "Audífonos", region %in% c("Norte", "Sur"
  ↪ ))
t.test(unidades ~ region, data = audif)

# 3
modelo_int <- lm(unidades ~ temporada * canal, data = semanal)
c(sin_interaccion = glance(modelo)$adj.r.squared,
  con_interaccion = glance(modelo_int)$adj.r.squared)
anova(modelo, modelo_int)

# 4 (ejemplo): "Fuera de noviembre-diciembre, cada canal vende en promedio
# alrededor de X unidades menos por semana que en temporada alta, manteniendo
# el canal constante; la diferencia es estadísticamente clara (p < 0.001)."
coef(modelo)["temporadaResto"]
```

7.4. Evaluación (sumativa parcial)

Informe breve (1 página) con: pregunta de negocio, prueba o modelo elegido y **por qué**, resultado con intervalo de confianza, verificación de supuestos y una limitación. Rúbrica: pertinencia del método (30 %), ejecución correcta (30 %), interpretación (30 %), comunicación (10 %).

7.5. Referencias verificadas

- **Video:** StatQuest with Josh Starmer. *Linear Regression in R, Step-by-Step* [Video]. YouTube. https://www.youtube.com/watch?v=u1cc1r_Y7M0 [EN] [fecha POR VERIFICAR]
- **Video:** StatQuest with Josh Starmer. *Linear Regression and Linear Models* [Lista de reproducción]. YouTube. <https://www.youtube.com/playlist?list=PLblh5JK0oLUIzaEkCLIUxQFjPIlapw8nU> [EN]
- **Web:** R Core Team. *An Introduction to R* — sección “Statistical models in R”. <https://cran.r-project.org/doc/manuals/r-release/R-intro.html> [EN] [OA]
- **Libro:** Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2.^a ed.). O’Reilly Media. ISBN 978-1-4920-9740-2. Español: <https://davidrsch.github.io/r4ds-es/> [ES] [OA] — caps. de análisis exploratorio como base para modelar.

Módulo 8 — Reportes reproducibles con Quarto y proyecto integrador

- **Duración estimada:** 1 h teoría + 4 h proyecto
- **Objetivos de aprendizaje.** Al finalizar, la persona podrá:
 1. **Crear** un documento Quarto (.qmd) que combine texto, código R y resultados.
 2. **Configurar** opciones de *chunks* (`echo`, `warning`, `fig-cap`) y formatos de salida (HTML, Word, PDF).
 3. **Organizar** un proyecto reproducible (estructura de carpetas, rutas relativas, `renv`).
 4. **Diseñar y producir** un análisis completo de principio a fin (proyecto integrador).

8.1. Contenidos

1. Por qué la reproducibilidad: el código *es* la documentación del análisis.
2. Anatomía de un .qmd: encabezado YAML, Markdown y *chunks* de código.
3. Opciones de *chunk* con `#|`.
4. Tablas con `knitr::kable()`; gráficos con leyenda.
5. Renderizar: botón **Render** en RStudio o `quarto render` en terminal.
6. Estructura de proyecto y control de dependencias con `renv`.
7. Guía de estilo del tidyverse; control de versiones con Git (introducción).

8.2. Desarrollo

8.2.1. Un documento Quarto mínimo

Guarda esto como `reporte.qmd` dentro del proyecto y presiona **Render**:

```
---
title: "Reporte de ventas 2025"
author: "Tu nombre"
format:
  html:
    toc: true
execute:
  warning: false
---

## Resumen

```{r}
#| label: datos
#| echo: false
library(tidyverse)
ventas <- read_csv("datos/ventas.csv", show_col_types = FALSE) |>
 mutate(ingreso = precio * unidades)
```

```
total <- sum(ventas$ingreso, na.rm = TRUE)
```


El ingreso total del año fue de **`r scales::dollar(total)`**.



```
```{r}
#| label: fig-producto
#| fig-cap: "Ingreso anual por producto"
ventas |>
  summarise(ingreso = sum(ingreso, na.rm = TRUE), .by = producto) |>
  ggplot(aes(ingreso, fct_reorder(producto, ingreso))) +
  geom_col() +
  labs(x = "Ingreso", y = NULL)
```
```


```

- ``r expresion`` inserta resultados **dentro del texto**: si cambian los datos, el número se actualiza solo.
- Cambia `format`: html por docx o pdf (este último requiere TinyTeX: `quarto install` ↪ `tinytex`).

8.2.2. Proyecto reproducible

```
mi-analisis/
|-- mi-analisis.Rproj
|-- datos/           # datos crudos: nunca se editan a mano
|-- R/               # funciones reutilizables
|-- reporte.qmd
`-- salidas/         # gráficos y tablas generadas
```

```
# Registrar versiones de paquetes del proyecto (una vez, en la consola):
# install.packages("renv"); renv::init()
# Después de instalar o actualizar paquetes:
# renv::snapshot()
```

8.3. Proyecto integrador (capstone)

Caso: Eres analista de una cadena de electrónica. Dirección pide un **reporte de desempeño 2025** con recomendaciones para la temporada alta de 2026.

Entregables:

1. Proyecto de RStudio con la estructura anterior.
2. `reporte.qmd` renderizado a HTML que incluya:
 - Limpieza documentada (faltantes, tipos, verificación de llaves).
 - Tabla de KPIs por canal y región (ingreso, unidades, ticket promedio, participación).
 - Al menos 3 gráficos con títulos que afirmen la conclusión.
 - Una prueba estadística o modelo con interpretación.
 - Al menos una función propia en `R/` usada en el reporte.
 - Conclusiones y 3 recomendaciones accionables.
3. Presentación oral de 5 minutos.

8.3.1. Rúbrica global del proyecto

Criterio (peso)	4 — Excelente	3 — Bueno	2 — Suficiente	1 — Insuficiente
Reproducibilidad (20 %)	Renderiza sin errores en otra computadora; rutas relativas; <code>renv</code>	Renderiza con ajustes menores	Requiere varios ajustes	No renderiza
Manejo de datos (20 %)	Limpieza completa y justificada; verificación de llaves	Limpieza correcta, poco documentada	Limpieza parcial	Datos sin revisar
Transformación y KPIs (20 %)	KPIs correctos, código <code>dplyr</code> idiomático	Correctos con código redundante	Errores menores	Errores graves
Visualización (15 %)	Gráficos adecuados, claros y con mensaje	Adecuados, mejorables	Tipo de gráfico poco apropiado	Ilegibles
Estadística (15 %)	Método pertinente, supuestos revisados, interpretación correcta	Pertinente, interpretación parcial	Método dudoso	Ausente o incorrecto
Comunicación (10 %)	Conclusiones y recomendaciones claras y accionables	Claras pero generales	Confusas	Ausentes

8.4. Referencias verificadas

- **Web:** Posit / Quarto. *Tutorial: Computations* (RStudio). <https://quarto.org/docs/get-started/computations/rstudio> [EN] [gratuito]
- **Web:** Posit / Quarto. *Using R*. <https://quarto.org/docs/computations/r.html> [EN] [gratuito]
- **Web:** Posit / Quarto. *Tutorial: Hello, Quarto*. <https://quarto.org/docs/get-started/hello/rstudio.html> [EN] [gratuito]
- **Libro:** Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2.^a ed.). O'Reilly Media. ISBN 978-1-4920-9740-2. Español: <https://davidrsch.github.io/r4ds-es/> [ES] [OA] — caps. “Quarto” y “Formatos de Quarto”.
- **Artículo:** Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10> [EN] [OA]